COMP11124 Interactive Software Design

Introduction to Python Programming I

Learning Outcomes

The practical lab exercises in this document serve as an introduction to Python. After completing this week's class, you should be able to implement your first own Python procedural programs, understand how to use the Python syntax.

Topics Covered

Visual Studio Code, Running Python Code, Variables, Operators, Print, Casting, Strings, f-Strings

---

Section 1: Setting Up a Code Editor and Running Python Scripts

The lab machines should be ready and installed with visual studio code and python. Do not install programmes on these machines.

If you wish to work on your personal PC at home you should first of all set up the core module software. If you are using lab machines you can skip this installation section.

Exercise 0: Install Python on a personal machine

We first need to install the Python interpreter before we start with any programming in Python.

Task: Download the python install manager: https://www.python.org/downloads/
Run the install manager and select the Install python button.

It is important to add the dirctory holding python to the PATH variable so that the python interpreter can be found by other programmes. Note that the directory "… AppData\Local\Python\bin" will contain the python executable. AppData is normally a hidden directory and will not normally be visible unless you set file explorer to view hidden files.

In the lab python is installed in a different location C:/Program Files/Python314/python.exe

Answer "y" when prompted.

```
Welcome to the Python installation manager configuration helper.

********************************************************************************

The global shortcuts directory is not configured.

Configuring this enables commands like python3.14.exe to run from your
terminal, but is not needed for the python or py commands (for example, py
-V:3.14).

We can add the directory (C:\Users\derek\AppData\Local\Python\bin) to PATH now,
but you will need to restart your terminal to use it. The entry will be removed
if you run py uninstall --purge, or else you can remove it manually when
uninstalling Python.
Add commands directory to your PATH now? [y/N]
```

Answer “Y” to install the python runtime when prompted for Cpython

```
********************************************************************************

You do not have the latest Python runtime.

Install the current latest version of CPython? If not, you can use 'py install
default' later to install.

Install CPython now? [Y/n]
```

When installation is complete you can select “y” to view online help and be taken to a section of the python docs.

The general python documentation can be found at https://docs.python.org/dev/index.html

Exercise 1: Setting Up Visual Studio Code Editor

To write any Python code we will first need a code editor. In this module, we will use the Visual Studio Code (VS Code) editor developed by Microsoft. This code editor does not only support Python, but it also supports reading and editing a range of other file types and programming languages.

Task: Download and install VS Code on your computer. You can find the link to download it here: https://code.visualstudio.com/ It would also be beneficial if you have a look around on the linked site, as it also contains information about other features of this editor.
Note: If you are using a university lab computer, you can skip this step as the editor is already be installed.

Once you have installed it, you can now start the editor.

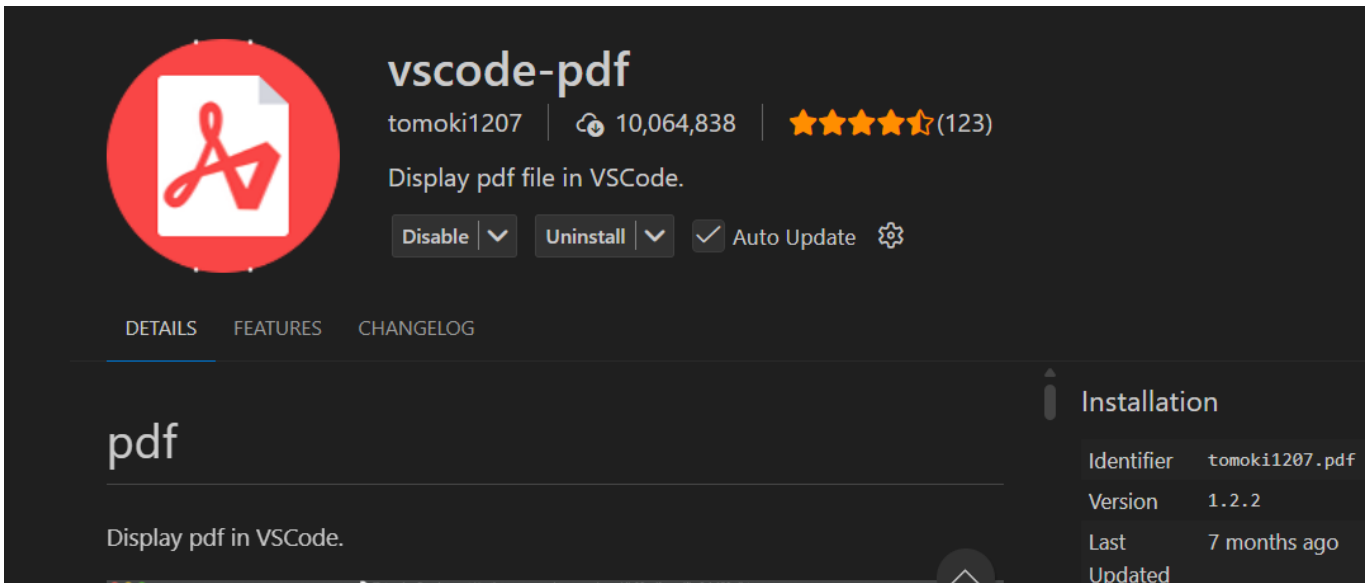
Visual Studio Code supports a number of extensions for a wide range of languages. The extensions which are useful for this module may not be added to the lab machines yet so you should check and install these whether you are using a personal or a lab machine.

Task: Start the editor and select the icon to see the extensions. In the search bar enter “python” and from the list which opens select and install Python extension for visual studio code.

Search for and install the Jupyter extension

These will be usefull straigh away. For future weeks also add the extension Markdown PDF. This will enable log book notes which will be developed to be output to pdf format.

Also add vscod-pdf which will allow pdf files to be viewed within the visual studio code editor.



Task: Download a starter folder to work in and store this on your machine. Remember to back files up before you leave the lab. A folder with a structure ready for the labsheet exercises can be downloaded from Aula. Please use this, maintaining an organised file structure is essential as the number and length of program files grows. The folder has a starter file in Sessions|session_1|lab_week1.py

Task: Start the editor and open the starter file for this week available on aula (*lab_week1.py*).

Run the program with the icon.

This may work first time, skip the next section if it does, however, in a new set Visual Studio Code may not know where to find the Python executable and generate an error "… python.exe is not recognized …"

You may see a message to the bottom right of the VSC screen asking you to select the python interpreter. If you have updated the version of python you are using on a personal machine, VSC may be left pointing at an empty location. In either case you can use CTRL + SHIFT + P to see a drop down list

of available commands. Choose “Python: Select interpreter” then “Enter interpreter path” then “Browse your file system to find a Python Interpreter and select an option such as python3.14.exe.

If you look around in the VSC editor, you will see the Python version that is currently installed on the bottom right-hand corner:

A note on AI. You may see the github copilot icon in visual studio code, now or as you work through the module. Don’t use this or any other AI coding agent. These are super useful tools when you know what to do with them and have a basic understanding of programme language and structure. However at this stage of learning they will impede the learning process by suggesting solutions which bypass your thought process and do not lead to the correct solutions to the lab exercises which are quite specific. When you have passed this module, that should not be the end of your programming journey, and investigating AI tools should be something to do at that point.

Task: Click the play button at the top right hand corner and look at the output of the script. Change the value of the variable “class*_name”* and run the script again. Does the output change?

What we have done here is running the script using the editor and that will be the normal process for this module.

Exercise 2: Running a Script Using the Command Line

As well as running a script in the editor we can use the command line to run our Python scripts as well. To do so we need to open the command line tool (which is CMD on Windows and Terminal on Mac).

Task: Open the command line tool on your computer. Navigate to the directory where you have saved the *lab_week_1.py* file ***in the command line.*** The easiest way to do is to type the *cd* command and then follow this up with the directory where the file is saved in.

Example:
For example, I have saved my file in the *Documents* directory. Therefore, after opening the command line I would type: cd Documents. On windows you can use the dir command to view the contents of the current folder.

Navigate on your own machine to “…\Python-ISD26\Sessions\session_1

Once we have changed the directory successfully, we want to type the command to run the file.
It is either: ***python lab_week_1.py*** *or* ***py lab_week_1.py*** depending on the installation detail.

You should now see the same output as when you have run the file via VSCode.

You should now understand that there are multiple ways how you can run Python files, both ways are valid and usually give the same output. Close the command line and return to VSC.

Section 2: Python Introduction

Exercise 1: Variables and Types

A variable is a named location in memory used to store data. You can assign a value to a variable using the assignment operator **=** . Variables are fundamental in programming and play a key role in writing dynamic and flexible code. Variables allow you to work with data by giving it a name.
Python supports various data types, including integers, floats, and strings. In this exercise you will practice working with different data types and find out how 1) we can use the type() function to check the data type of a variable, 2) we can convert (cast) between different data types using the int(), float(), and str() functions, and 3) we can use the print() function to print out the data type of a variable. A variable is a named location in memory used to store data.

Below is some example code that shows you how variables in Python are used. Here we create variables and assign them certain values.

```
# Create a variable 'x' and assign it the value 10
x = 10

# Create a variable 'name' and assign it a string
name = "Alice"

# Create a variable 'pi' and assign it a float value
pi = 3.14159
```

Task 1: Create a new Python file called *lab_week1_part2.py inside the session_1 folder*. To do this click on File -> New File in VS code and store the file using this name. This should also automatically open the file.

Task 1: Copy the following code into your new Python file:

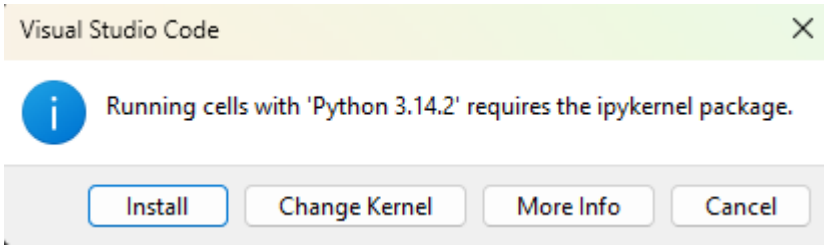
```
# Session 1 Section 2 Python Introduction

Run Cell | Run Below | Debug Cell
#%%
#Exercise 1 task: Variables and Types

var_1 = True #Type =
var_2 = 1 #Type =
var_3 = 3.14159 #Type =
var_4 = "Hello World" #Type =
```

Then, look at the value of each variable and try to guess what type they are. Put your guess behind the "equals = " sign into the code. If you are unsure, use the following link to help you:
https://www.w3schools.com/python/python_datatypes.asp

Task 1: To now double check whether your guesses where correct we can use Python's built in *type()* function and print out the result.
Use the following syntax:

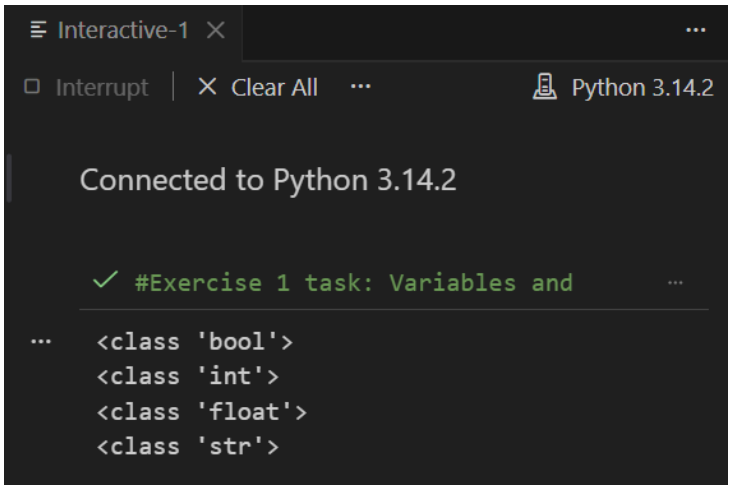
```
print(type(var_1))
```

Click on Run Cell ro run the programme. You may see a prompt:



If so do install. This intalls just enough of the Jupyter application to allow visual studio code to work with interactive python.

When you run a cell an interactive window opens alongside your code.



Apply this to each or the variables to print out the type for each variable (var_1 to var_4). Does this align with your initial guesses?

The interactive display is useful for combining a group of small programme fragments into a single file and allowing each fragment to run in its own cell. The output is neater than running the full file to a single output.

The cell can be extended to show both code and output by clicking on "…" after the first line. The output can be copied by clicking on "…" on the second line.

```
#Exercise 1 task: Variables and Types

var_1 = True #Type = boolean
var_2 = 1 #Type =   integer
var_3 = 3.14159 #Type = float
var_4 = "Hello World" #Type = string

print(type(var_1))
print(type(var_2))
print(type(var_3))
print(type(var_4))
```

[1] ✓ 0.0s

```
<class 'bool'>
<class 'int'>
<class 'float'>
<class 'str'>
```

The print function is very helpful for finding out what values are stored in variables and what type they are. There are better ways to do this, but for now, this is a good way to check your work.

At last, Python also allows us to convert values from one type to another. This is called casting. We can cast a value to a string by using the str function like this: str(5).

Other functions that we can use are:
- int() to convert to an integer
- float() to convert to a floating point number
- bool() to convert to a Boolean value

Task 2:

Setup another cell below the code already written in lab_week1_part2.py

```
print(type(var_1))
print(type(var_2))
print(type(var_3))
print(type(var_4))

Run Cell | Run Above | Debug Cell
#%%
#Exercise 1 task 2: Casting Variables
```

In this cell write code following these steps:

1. Create a variable called my_int and assign it the value 5.
2. Create a variable called my_float and assign it the value 5.5.
3. Create a variable called my_bool and assign it the value True.
4. Print the value of each variable.
5. Cast my_int to a float and store the result in a variable called my_int_float.
6. Cast my_float to an integer and store the result in a variable called my_float_int.
7. Cast my_bool to an integer and store the result in a variable called my_bool_int.
8. Print the value of each variable.

Then observe the output of each variable. Does this algin with what you thought the variables will be after casting? To find out some more information, it is recommended to check out this link: https://www.w3schools.com/python/python_casting.asp

Exercise 2: Arithmetic Operators

Python arithmetic operators, which allow you to perform mathematical operations on numeric values. Understanding and using these operators is fundamental as they make up the base of a lot of programs.

Python provides several arithmetic operators for performing basic mathematical operations:

- **Addition (`+`):** Adds two values.
- **Subtraction (`-`):** Subtracts the right operand from the left operand.
- **Multiplication (`*`):** Multiplies two values.
- **Division (`/`):** Divides the left operand by the right operand.
- **Floor Division (`//`):** Performs floor division, returning the quotient as an integer.
- **Modulus (`%`):** Returns the remainder of the division.
- **Exponentiation (`**`):** Raises the left operand to the power of the right operand.

Task 1: Copy and paste the following code into your Python file, try to understand it, and then run it:

```python
#%%
# Exercise 2 Arithmetic operators
#Addition
result_addition = 10 + 5
print("Addition:", result_addition)

#Subtraction
result_subtraction = 20 - 8
print("Subtraction:", result_subtraction)

#Multiplication
result_multiplication = 6 * 4
print("Multiplication:", result_multiplication)

#Division
result_division = 15 / 3
print("Division:", result_division)

# Floor Division
result_floor_division = 17 // 4
print("Floor Division:", result_floor_division)

#Modulus
result_modulus = 17 % 4
print("Modulus:", result_modulus)

#Exponentiation
result_exponentiation = 2 ** 3
print("Exponentiation:", result_exponentiation)
```

You will also see that we are using the *print* function not only to print out one value, but several values.

Now, let us put the use of those operators and variables into practices by performing two common kinds of calculations. Use a separate shell for each of the following tasks.

Task 2: Calculating the Average:

1. Create two variables, *num1* and *num2*, with any numeric values.
2. Calculate their average using the arithmetic operators and assign it to a variable called *average*.
3. Print a message that displays the values of num1, num2, and the calculated average.

Task 3: Calculate the Area of a Rectangle:

1. Create two variables, *length* and *width*, representing the dimensions of a rectangle.
2. Calculate the area of the rectangle using the arithmetic operators and assign it to a variable called *area*.
3. Print a message that displays the values of length, width, and the calculated area.

Exercise 3: Strings and f-Strings

So far you will have used the String data type to store sequences of characters in variables. However, Python offers us also extra functionality that we can use with strings. Each variable that contains a string is an Object (more on that in later classes) and has built in methods (that are actions that it can perform).
You can find a list of methods that you can perform on strings in this documentation:
https://www.w3schools.com/python/python_ref_string.asp

As an aspiring programmer we do not expect you to know everything about a programming language. One very important soft-skill is to be able to use documentations (that exist for every programming language). You need to be able to find a way to look for a solution, even if you have not done it before. In that regard, the next task requires you to use the link above and modify string-variables to do the following:

Task 1: Modify Strings

1. Create a new variable called *my_string* and store the following in it: "*This class covers ISD*."
2. Print the *my_string* variable
3. Using the documentation in the link above, look through the examples and do the following:
    a. Create a new variable *my_uppercase_string* that stores the *my_string* converted to uppercase
    b. Create a new variable *my_lowercase_string* that stores the *my_string* converted to lowercase
    c. Create a new variable *my_new_string* that replaces the word "ISD" in the string with "Interactive Software Design".
    d. Create a new variable *my_string_length* that stores the length of the string (Note: use a search engine or the lecture slides to help you how to do this).

F-Strings (also called formatted string literals) allow you to evaluate expressions within a string. When you create a new string, you will need to preface it with the letter ***f*** and can then evaluate your expressions within the quotes of a string if you limit them by curly brackets **{expression}**.

This can allow us to write less code if we want to combine multiple variables into a string for example. Consider the following:

```
class_name = "ISD"
number_of_students = 20
my_string = "Welcome to the " + class_name + " module. There are " + str(number_of_students) + "
students in this class."
```

You will need to write parts of the string, and then concatenate them with other values to create your output "my_string".
F-Strings make this much easier, the code above can be converted simply to:

```
my_string = f"Welcome to the {class_name} module. There are {number_of_students} students in this
class."
```

and will yield the same result. Note the *f* at the beginning before the opening quotation and the variable names wrapped in {}.

Task 2: f-Strings

1. Create variables called *my_name*, *number_of_classes*, *campus*. Store your name, the number of classes you are taking this term and then the campus you are studying at.
2. Create an f-string using the instructions above called *my_text* that combines the text to the following scheme. (Example: my_name = Peter, number of classes = 10, campus = Paisley):
   "My name is Peter and I am studying 10 classes in Paisley".
3. Print the string, run the program, and ensure that you have done everything correctly.

Section 2: First Program in Python

In this exercise, you will write a Python program that allows the user to convert temperatures from Celsius to Fahrenheit and Kelvin.

Your **Task** is to look at the following requirements and implement the program in the file called *temperature_converter.py*.

Requirements:

- The user stores the Celsius value as a number in a variable called *celsius_input*
- The Celsius value then gets converted to Fahrenheit and stored in a variable called *degree_f*
- The Celsius value then gets converted to Kelvin and stored in a variable called *degree_k*
- (Note: Use the internet to find out the formulas on how to convert C to K and F and implement that using Python operators
- Generate and print an output that follows the following format:

Welcome to the Temperature Converter!

The temperature you have entered is ***{USER ENTERED C}*** *degree Celsius.*

Converted Temperatures:
{USER ENTERED C} *degree Celsius is equal to* ***{COMPUTED degree_f}*** Fahrenheit.
{USER ENTERED C} *degree Celsius is equal to* ***{COMPUTED degree_k}*** Kelvin.

Thank you for using the Temperature Converter!

Once you are finished with this task, please show the output of this and the code to your lab tutor for formative feedback.